

Использование StringView / String / StackString

Предусмотрено три основных класса для работы со строками StringView / String / StackString. В данной статье разобрано в какой ситуации какой использовать предпочтительнее. Подробнее о работе со строками описано в статье ["Работа со строками"](#).

Использование String

String необходимо использовать, если метод подразумевает копирование строки в класс. Объектами класса **String** следует пользоваться, если вызываемая функция или метод класса подразумевает копирование переданной строки в ее исходном виде. Как правило, это всевозможные методы set-теры, а также конструкторы или фабричные функции, ответственные за правильное создание объектов.

Например:

```
1 class Foo
2 {
3 public:
4     void SetName(String name )
5     {
6         mName = std::move ( name );
7     }
8 private:
9     String mName;
10 }
```

В данном примере происходит передача значения строки через семантику перемещений (std::move), что позволяет избежать лишних операций.

Примеры

```
1 // Функция GetStringReference() возвращает ссылку на строку.
2 foo.SetName( GetStringReference() );
```

При вызове метода ссылка копируется в аргумент. Аргумент перемещается в объект. Одно копирование, одно перемещение.

```
1 foo.SetName( L"Some name" );
```

Вызов метода от строки константы. Создается аргумент строка и помещается в объект.

```
1 String str = fill();
2 foo.SetName( str );
3 SomeAnotherFn( str );
```

В случае если существует строка, которую нельзя забрать. При вызове метода создается копия и помещается в объект.

```
1 foo.SetName( GetStringByValue() );
```

Если существует временная строка при вызове метода запускается move - конструктор и параметр конструируется не копированием, а перемещением. В метод попадет именно та строка, которую вернула GetStringByValue. В данном случае есть два перемещения и нет копирования.

Например, необходимо передать значения логина и пароля в объект User.

Пример конструктора класса:

```
1 class User
2 {
3     String mLogin;
4     String mPassword;
5     public:
6     User( String login, String password ) noexcept
7         : mLogin( std::move( login ) ),
8           mPassword( std::move( password ) )
9     {}
10 };
```

Использование объекта User

```
1 // Использование
2 User user{ L"login", L"password" };
```

Создаются временные строки по строковым константам и в списке инициализации конструктора перемещаются в поля класса.

Использование StringView

Объект класса **StringView** следует использовать, если функция/метод принимает строку только для чтения. Сюда же относятся ситуации, когда функция/метод будет производить какую-то дополнительную обработку переданной строки.

Пример

```
1 class DbStatement
2 {
3     public:
4         void Exec( StringView query )
5         {
6             internal_c_driver_exec( query.data(), query.size() );
7         }
8 };

1 stmt.Exec( GetQuery() );
```

Явный вызов StringView.

На этапе компиляции можно создавать константу StringView

Функция ParseLoginPwd из StringView строит объект User, при этом не изменяется исходная строка, не создается временных строк и нет аллокации памяти до момента создания объекта.

Пример

```
1 // строка login_pwd имеет формат логин:пароль и не пригодна для
  // немедленного построения
2 // по ней объекта типа User
3 User ParseLoginPwd( StringView login_pwd )
4 {
5
6     // Split возвращает пару {StringView,StringView}, данные которых
```

```

    смотрят на login_pwd
7     auto [ login,pwd ] = Split( login_pwd, L":" );
8
9     // в точке конструирования объекта мы делаем String из StringView и
    передаем владение ими объекту
10    return { login.to_string(), pwd.to_string() };
11 }

```



Рекомендуется передавать StringView по значению, т.к. вызов по ссылке приводит к протечке аргумента на стек вызывающей функции, косвенному обращению и ограничению на оптимизацию псевдонимов в вызываемой функции.

Ограничения StringView

- Не позволяет модифицировать данные, поскольку StringView, по своей природе, ссылка read-only.
- Содержит ненультерминированную строку, вследствие этого: нет метода `c_str`, нельзя использовать результат `data()` как нультерминированную строку.
- Нельзя хранить StringView, ссылающийся на данные, которые могут "умереть" раньше его. Если необходимо долгосрочно запомнить данные, переданные в метод в виде StringView, необходимо их копировать в какой-то контейнер (`StackString`, `String`).

Использование StackString

Объектами класса **StackString** следует пользоваться в том случае, когда необходимо создать из составляющих короткоживущую строку, передать ее в некую функцию/метод только на чтение, а затем освободить.

Пример

```

1 void DoHttpRequest( StringView domain, int api_version, StringView
    method, StringView params )
2 {
3     StackString< 128 > url;
4     Format( url, L"http://%1%/api_v%2%/%3%/?"_sv, domain,
    api_version, method, params );
5     DoHttpRequest( StringView{ url } );
6
7     // здесь url будет удален и его данные более никому не нужны
8 }

```